

Než začneš psát cykly a pracovat se seznamy, zopakuj si průměr a četnosti, opakování v algoritmu, přiřazení do proměnné, větvení a testování na hraničních případech. Cykly z nich skládají celé programy.

1. Průměr, maximum a četnosti ZŠ

Kde to najdeš: Začátečníci H23 Tabulky, diagramy a průměr

Aritmetický průměr je součet všech hodnot vydělený jejich počtem. Maximum je největší hodnota, minimum nejmenší. Četnost hodnoty říká, kolikrát se v datech vyskytuje; relativní četnost je tento počet vydělený počtem všech hodnot (často v procentech). Modus je hodnota s největší četností. Přesně tyto výpočty budeš programovat cyklem.

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_n}{n} \text{ aritmetický průměr}$$

Příklad: Anketa „Kolik dní v týdnu sportuješ?“ dala odpovědi 2, 3, 3, 5, 0, 3, 5. Urči průměr, maximum a četnosti.

1. Součet: $2 + 3 + 3 + 5 + 0 + 3 + 5 = 21$, počet hodnot 7.
2. Průměr: $\frac{21}{7} = 3$ dny; maximum je 5.
3. Četnosti: $0 \rightarrow 1\times$, $2 \rightarrow 1\times$, $3 \rightarrow 3\times$, $5 \rightarrow 2\times$; modus je 3.

Zkus sám:

1. Urči průměr čísel 4, 8, 15, 16, 23, 42.
2. Jaká je relativní četnost hodnoty 3 v datech 1, 3, 3, 2, 3, 1, 2, 3?

2. Opakování v algoritmu SŠ

Kde to najdeš: 1.+2. H4 Co je algoritmus, 1.+2. H6 Zápis algoritmu

Algoritmus často obsahuje krok, který se opakuje: „dokud není hrnec plný, přilij hrnek vody“. Opakování má dvě podoby: buď předem víš, kolikrát se krok provede (třikrát zamíchej), nebo se opakuje, dokud platí podmínka. Ve vývojovém diagramu vypadá cyklus jako šipka vracející se k podmínce. V Pythonu první případ zapíšeš cyklem for, druhý cyklem while. U opakování s podmínkou musíš zajistit, že podmínka jednou přestane platit – jinak cyklus neskončí.

Příklad: Máš 100 Kč a každý den utratíš 30 Kč, dokud na to máš. Kolikrát nakoupíš a kolik ti zbude?

1. Opakuj, dokud je zůstatek aspoň 30 Kč – to je opakování s podmínkou (while).
2. Zůstatek se mění $100 \rightarrow 70 \rightarrow 40 \rightarrow 10$.
3. Nakoupíš třikrát a zbude 10 Kč; při zůstatku 10 Kč podmínka neplatí a opakování skončí.

Zkus sám:

1. Kolikrát se provede krok „vyděl číslo dvěma“ pro číslo 64, dokud není rovno 1?
2. Je „zamíchej pětikrát“ opakování s pevným počtem, nebo s podmínkou?

3. Proměnná, přiřazení a akumulátor SŠ

Kde to najdeš: 1.+2. H41 První program

Přiřazení $s = s + x$ znamená: vezmi starou hodnotu s , přičti x a výsledek ulož zpátky do s . Není to rovnice – v matematice by $s = s + x$ platilo jen pro $x = 0$. Takto se v cyklu postupně sčítají čísla; proměnné, která si průběžný výsledek pamatuje, se říká akumulátor. Počáteční hodnotu (pro součet 0) musí akumulátor dostat ještě před cyklem.

Příklad: Co vypíše program $s = 0$, $s = s + 4$, $s = s + 7$, $s = s + 1$, $\text{print}(s)$?

1. Na začátku $s = 0$.
2. Po prvním přičtení $s = 4$, po druhém 11, po třetím 12.
3. Program vypíše 12.

Zkus sám:

1. Co vypíše program $p = 1$, $p = p * 2$, $p = p * 3$, $p = p * 4$, $\text{print}(p)$?
2. Proč akumulátor pro součin začíná hodnotou 1, a ne 0?

4. Větvení uvnitř programu SS

Kde to najdeš: 1.+2. H44 Větvení: řešič rovnic

Uvnitř cyklu často potřebuješ rozhodnout, co s prvkem udělat: přičíst ho jen tehdy, když je kladný, započítat ho do správné kategorie, nebo si ho zapamatovat jako nové maximum. K tomu slouží if, elif a else, stejně jako v řešiči kvadratické rovnice. Tělo cyklu i tělo větve se poznají podle odsazení: čím je příkaz víc odsazený, tím hlouběji je vnořený.

Příklad: Pro seznam [3, -1, 4, -5, 2] spočítej součet jen kladných čísel tak, jak to udělá cyklus s podmínkou if x > 0.

1. 3 je kladné → součet 3.
2. -1 přeskočím, 4 přičtu → 7.
3. -5 přeskočím, 2 přičtu → 9.
4. Výsledek je 9.

Zkus sám:

1. Kolik čísel ze seznamu [5, 12, 7, 20, 3] je větších než 6?
2. Algoritmus maxima ukládá nové maximum jen při x > nejvic. Kterou hodnotu si zapamatuje pro [4, 9, 2, 9, 1]?

5. Hraniční případy a testování SS

Kde to najdeš: 1.+2. H47 Testování a ladění

Program, který zpracovává seznam, musíš otestovat i na neobvyklých datech: prázdný seznam, seznam s jediným prvkem, samá záporná čísla nebo opakující se maximum. Průměr prázdného seznamu skončí chybou ZeroDivisionError a maximum „od nuly“ selže u záporných čísel. Výsledek svého cyklu vždy porovnej s vestavěnou funkcí (sum, max, len) – takový test ti chybu prozradí.

Příklad: Algoritmus maxima začíná nejvic = 0. Co vrátí pro seznam [-3, -8, -1] a jak ho opravit?

1. Žádný prvek není větší než 0, proto nejvic zůstane 0.
2. Správné maximum je -1 – program se mýlí.
3. Oprava: začni prvním prvkem, nejvic = s[0].
4. Test: výsledek porovnej s max(s).

Zkus sám:

1. Co se stane při výpočtu sum(s) / len(s) pro s = []?
2. Jaký výsledek má vrátit hledání maxima pro seznam [7]?

Kde se to doučit

- **Seznamy – Nauč se Python (PyLadies)** – Vytvoření seznamu, indexy, přidávání prvků a procházení seznamu cyklem for.
<https://naucese.python.cz/course/pyladies/beginners/list/>
- **Cyklus while – Nauč se Python** – Opakování, dokud platí podmínka, a rozdíl mezi cykly while a for.
<https://naucese.python.cz/course/pyladies/beginners/while/>
- **Python Tutor – vizualizace běhu programu** – Vložíš krátký program a krokuješ ho řádek po řádku; vidíš, jak se mění hodnoty proměnných, seznamů a volání funkcí.
<https://pythontutor.com/visualize.html>

Výsledky ke kontrole

- **1. Průměr, maximum a četnosti:** 1) 18; 2) $\frac{4}{8} = 50\%$
- **2. Opakování v algoritmu:** 1) šestkrát ($64 = 2^6$); 2) s pevným počtem – v Pythonu cyklus for
- **3. Proměnná, přiřazení a akumulátor:** 1) 24; 2) 0 krát cokoli je 0 – výsledek by byl vždy 0
- **4. Větvení uvnitř programu:** 1) 3 (12, 7 a 20); 2) 9 (první devítku; druhá už podmínku nesplní)
- **5. Hraniční případy a testování:** 1) ZeroDivisionError – dělení nulou; 2) 7